

Distributed Multi-Sensor Control for Multi-Target Tracking

Aidan Blair, Amirali Khodadadian Gostar, Ruwan Tennakoon, Alireza Bab-Hadiashar, Xiaodong Li,
Jennifer Palmer and Reza Hoseinnezhad

RMIT University
Melbourne, Australia

Abstract—This paper proposes a new sensor control algorithm for multi-target tracking applications within distributed sensor networks. In multi-target tracking applications, most sensor control algorithms are designed for centralized sensor networks, where there is a central processing node that is computationally inefficient. This paper first provides a conceptual and mathematical overview of the multi-sensor multi-target tracking framework, using random finite set (RFS) filters and sensor fusion. We will also provide an overview of the existing sensor control methods. We then explore coordinate descent-based sensor control and introduce a fully distributed algorithm utilizing coordinate descent and an information-theoretic objective function. This method is tested on synthetic data and compared to alternative methods. The results show that the proposed method outperforms equivalent independent multi-sensor control methods.

Index Terms—distributed sensor network, sensor control, coordinate descent, random finite sets, multi-target tracking

I. INTRODUCTION

Intelligently controlling a large network of autonomous sensor-carrying vehicles is of significant interest in various fields. Applications of multi-target tracking within distributed sensor networks include autonomous vehicle network control [1], surveillance and intelligence applications [2] and multi-target tracking [3].

Sensor networks can be broadly split into three categories, centralized, decentralized and distributed [4], [5]. Centralized networks contain a central processing node, with communication between it and every other node in the network, that gathers all the information from the nodes, processes it and distributes it back out to the other nodes. In decentralized networks, nodes form local clusters, each with a central node that communicates with the other nodes in that cluster. These nodes can also communicate with other central nodes in the network. Distributed networks, by contrast, do not contain a centralized node. Instead, nodes can only communicate with their neighbors, usually within a certain communication range.

Centralized approaches to multi-sensor control work well for small clusters of sensors, but the exponential growth in required computational power with each added sensor prevents them from being used practically in these scenarios. Alternatively, distributed algorithms do not face these scaling problems. However they tend not to perform as well due to decisions being made without complete information. In this paper, we propose a distributed sensor control algorithm for multi-target tracking applications that is computationally

tractable and of comparable accuracy to centralized algorithms.

The aim of a sensor control algorithm is to select an action out of a set of possible actions that optimizes an objective function. The objective function can be one of two approaches: task-driven and information-driven [6]. The task-driven cost functions depend on the application. Examples include individual metrics such as state error variance [7] and cardinality error variance [8], as well as linear combinations of metrics like the Posterior Expected Error of Cardinality and States (PEECS) cost function [9]. In information-driven reward functions, the value associated with an action is the expected information gain from taking that action. Examples include Rényi Divergence, Cauchy-Schwarz Divergence, and Kullback-Leibler Divergence [10].

Random Finite Sets (RFS) is a Bayesian framework for multi-target filtering where the sets of target states and sets of observations are modeled as random finite sets. Many filters based on this framework have been developed, such as probability hypothesis density (PHD) [11] and multi-Bernoulli (MeMBer) [12] filters. RFS filters are an alternative to traditional methods such as Kalman filters. RFS filters are used in a variety of fields including cell lineage tracking [13], intelligent transport systems (ITS) applications [14] and information fusion [15], [16] in (multi)target tracking applications.

Sensor control methods have been proposed for a variety of RFS filters via different objective functions [17]–[20]. However, these methods have been mainly explored in single-sensor control scenarios. Multi-sensor control for labeled random finite set filters has been introduced recently in works such as Jiang et al.'s [21] using a Cauchy-Schwarz-based Divergence objective function in combination with Vo-Vo filters, and PEECS-based control using Labeled Multi-Bernoulli (LMB) filters [22]. Additionally, Panicker et al. [23] developed a multi-sensor control solution using LMB filters that focuses on *targets of interest*. Later an information-theoretic objective function for RFS-based multi-sensor control was proposed [24].

In this paper, we propose a distributed sensor control algorithm that utilizes an information-theoretic reward function and principles from coordinate descent, providing comparable performance to a centralized sensor control algorithm at a significant reduction of computing power.

In Section II the mathematical framework of multi-sensor,

TABLE I: Notations

Notation	Definition
x	single object state.
$\mathbf{X}$	multi-object state set.
$\mathbf{Z}$	measurement set.
$\pi(\cdot)$	a multi-object RFS density.
$\pi(\cdot)$	a labeled multi-object RFS density.
u	single-sensor control command.
$\mathbf{u}$	multi-sensor control command.
$D_{KL}(\cdot\ \cdot)$	Kullback-Leibler divergence between two densities.

multi-target control is presented, along with the RFS filter used. In Section III the proposed method and contributions are explained in detail. Section IV outlines the simulation scenario that was used to test the proposed method, the results and an analysis of the results. Finally Section V provides some conclusions on the proposed method and the obtained results.

II. MULTI-SENSOR CONTROL BACKGROUND

This section briefly summarizes some notation and concepts in the random finite sets framework and sensor control, that are essential to follow the rest of the paper.

A. Notations

Consistent with the recent literature on labeled RFS filters and sensor control for multi-object tracking, we used the notations defined in Table I

B. Multi-Target Tracking Problem Overview

For a multi-object system presented in discrete time k the multi-object state $\mathbf{X}_k$ can be described by a labeled random finite set (RFS) composed of several single-object states,

$$\mathbf{X}_k = \{(x_{1,k}, \ell_1), \dots, (x_{N_s,k}, \ell_{N_s})\} \in \mathcal{F}(\mathbb{X} \times \mathbb{L}) \quad (1)$$

where each state x has been augmented with a label ℓ . A network of sensors can be described as a graph $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$, where the vertices $\mathcal{V}$ represent the sensors and the edges $\mathcal{E}$ represent the connection between sensor nodes. At each discrete time step k , the multi-object state are partially detected by a cluster of N_s sensors, each returning a set of measurements $Z_i, i = 1 : N_s$ which is a subset of the measurement space, $Z \in \mathcal{F}(\mathbb{Z})$. Each sensor can be controlled via a sensor control command or one of a set of actions $u_i \in \mathbb{U}$, where $\mathbb{U}$ is a finite set of sensor control commands. A multi-sensor control command can be constructed by appending multiple single-sensor control commands, $\mathbf{u} = (u_1, \dots, u_{N_s}) \in \mathbb{U}^{N_s}$. The goal of controlling a set of sensors for multi-object tracking is to enhance and increase the accuracy of target state estimation both for the number of the targets and their states.

C. Multi-Target Bayes Filter

The probability density of a multi-object state can be recursively estimated via a multi-target Bayes filter. The density of a multi-object state at time k is denoted by $\pi_k(\mathbf{X}_k | \mathbf{Z}_{1:k})$, where $\mathbf{Z}_{1:k}$ is the set of all multi-sensor measurements accumulated up to time k . In Bayesian filtering, the density goes through

two steps at each time k , prediction and update. The predicted density is computed using the Chapman-Kalmogorov equation,

$$\pi_{k|k-1}(\mathbf{X}_k | \mathbf{Z}_{1:k-1}) = \int \pi_{k-1}(\mathbf{X}_{k-1} | \mathbf{Z}_{1:k-1}) \mathbf{f}(\mathbf{X}_k | \mathbf{X}_{k-1}) \delta \mathbf{X}_{k-1} \quad (2)$$

With new measurements $\mathbf{Z}_k = (Z_1, \dots, Z_{N_s})$ at time k after actions taken by the sensors informed by a multi-sensor control command $\mathbf{u}_k$, the posterior density is computed from the predicted density via the multi-object Bayes rule,

$$\pi_k(\mathbf{X}_k | \mathbf{Z}_{1:k}) = \frac{g(\mathbf{Z}_k | \mathbf{X}_k, \mathbf{u}_k) \pi_{k|k-1}(\mathbf{X}_k)}{\int g(\mathbf{Z}_k | \mathbf{X}, \mathbf{u}_k) \pi_{k|k-1}(\mathbf{X}) \delta \mathbf{X}} \quad (3)$$

A recent development in Multi-Target Bayes filters uses the random finite set (RFS) framework developed by Mahler [15] in which targets are treated as a set of random finite objects and each object representing an element of the set. Some of the RFS-based filters include the probability hypothesis density (PHD) filter [25], cardinalized PHD (CPHD) filter [26], multi-target multi-Bernoulli (MeMBer) [27], cardinality balanced MeMBer (CBMeMBer) [12], and the labeled multi-Bernoulli (LMB) filter [28]. A number of applications of target tracking used the RFS based framework to develop new solutions, track-before-detect visual tracking applications [29], sensor management in target tracking applications [23], [30] and information fusion [31]. In this paper we use LMB filter to develop a novel solution for the sensor control application.

D. Labeled Multi-Bernoulli Filter

The RFS filter used in this paper is the labeled multi-Bernoulli (LMB) filter [28]. The LMB distribution is completely described by its components $\pi = \{(r^{(\ell)}, p^{(\ell)}(\cdot))\}_{\ell \in \mathbb{L}}$ where $r^{(\ell)}$ is the probability of existence of an object with label $\ell \in \mathbb{L}$ and $p^{(\ell)}(x)$ is the probability density of an object state $x \in \mathbb{X}$ conditional on its existence.

The LMB density is

$$\pi((X)) = \Delta(\mathbf{X}) \mathbf{w}(\mathcal{L}(\mathbf{X})) [p]^{\mathbf{X}} \quad (4)$$

where $\mathcal{L}(\mathbf{X})$ is the set of all labels extracted from labeled states in $\mathbf{X}$ and

$$\Delta(\mathbf{X}) \triangleq \begin{cases} 1 & \text{if } |\mathbf{X}| = |\mathcal{L}(\mathbf{X})| \\ 0 & \text{otherwise,} \end{cases} \quad (5)$$

where $|\cdot|$ denotes the cardinality of the set,

$$[p]^{\mathbf{X}} \triangleq \prod_{(x, \ell) \in \mathbf{X}} p^{(\ell)}(x) \quad (6)$$

and

$$\mathbf{w}(L) = \prod_{i \in \mathbb{L}} (1 - r^{(i)}) \prod_{\ell \in \mathbb{L}} \frac{1_{\mathbb{L}}(\ell) r^{(\ell)}}{(1 - r^{(\ell)})} \quad (7)$$

is the probability of joint existence of all objects with label ℓ and non-existence of all other labels.

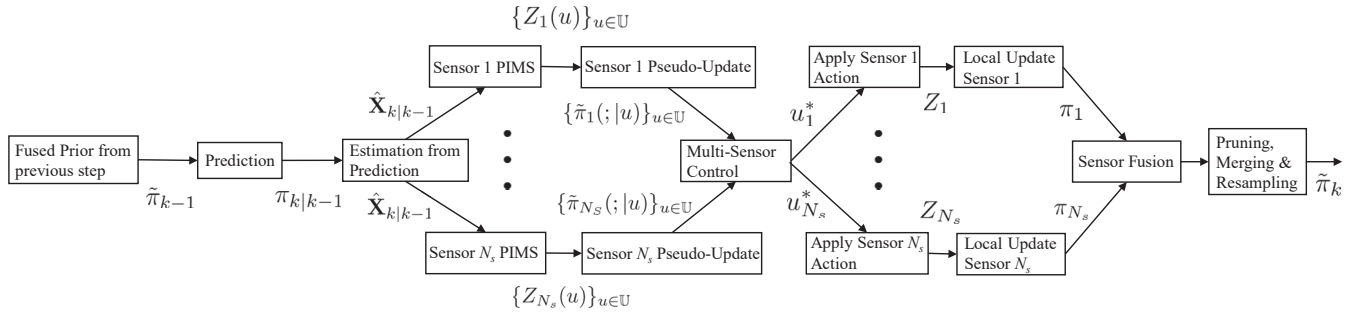


Fig. 1: Basic multi-sensor control block diagram using RFS framework.

In this paper a particle implementation is used, where the density of each LMB component with label ℓ is approximated by $J^{(\ell)}$ weights and particles,

$$p^{(\ell)}(x) \approx \sum_{j=1}^{J^{(\ell)}} w_j^{(\ell)} \delta(x - x_j^{(\ell)}) \quad (8)$$

where $\delta(\cdot)$ is the Dirac delta function.

E. Multi-Sensor Control for Multi-Target Tracking

The aim of multi-sensor control is to select the optimal multi-sensor control command $\mathbf{u}^* = (u_1^*, \dots, u_{N_s}^*)$, given the objective function $\nu(\cdot)$, where $\mathbf{u}^* = \arg \min/\max \nu(\mathbf{u}; \mathbf{X}_{k-1})$.

The multi-sensor control problem, using the RFS filtering framework, is demonstrated in Fig. 1. The fused prior density from the previous time step $\tilde{\pi}_{k-1}$ is passed through a Bayesian prediction step $\pi_{k|k-1}$ using (2). From this predicted density a set of estimates are generated $\hat{\mathbf{X}}_{k|k-1}$. At each sensor $i = 1 : N_s$ locally these estimates are used to generate sets of pseudo-measurements $\{Z_i(u)\}_{u \in \mathbb{U}}$ using the Predicted Ideal Measurement Set (PIMS) method [15] for each possible sensor action $u \in \mathbb{U}$. These pseudo-measurements are sets of hypothetical measurements that each sensor would receive after taking a specific action, assuming no measurement noise, no clutter and perfect detection of all targets. Each pseudo-measurement set undergoes a local update via (3), resulting in a pseudo-updated density $\{\pi(\cdot|u)\}_{u \in \mathbb{U}}$ for each possible action at each sensor.

With these pseudo-updated densities, multi-sensor control can be performed. After the control algorithm has determined the optimal sensor action u_i^* for each sensor, these are applied to the sensors. After the actions have been taken, measurement sets $\{Z_i\}$ are collected, the predicted densities are updated locally and the updated densities π_i are fused together. Post-processing including pruning, merging and resampling is performed to result in the fused posterior $\tilde{\pi}_k$ that will be the prior for the next time step, $\tilde{\pi}_{k+1}$.

III. METHODOLOGY

In this section, an overview of multi-sensor control framework is presented. At each time step k , at each sensor a prediction $\pi_{k|k-1}^{(i)}$ using the prior $\tilde{\pi}_{k-1}^{(i)}$ is calculated. A set

of object estimates $\hat{\mathbf{X}}_{k|k-1}$ is extracted from the predicted density, which is itself also calculated from the prior $\tilde{\pi}_{k-1}^{(i)}$. This multi-object estimate is used to compute a predicted ideal measurement set, PIMS [32], at each sensor node and for each possible control command $u \in \mathbb{U}$ applied at that node. Following that a pseudo-update is performed at each sensor using each PIMS associated with the possible control commands. The pseudo-posteriors $\{\tilde{\pi}_k^{(i)}\}_{i=1:N_s}$ are then used in an optimization method to determine which set of sensor control commands should be applied. These control commands are then applied by the sensors, and their state is updated before acquiring a new set of measurements $\{Z^{(i)}\}_{i=1:N_s}$. The measurements are used to update each sensor's posteriors locally. Then the local posteriors are fused using complementary information fusion [33], processed and become the new prior densities for the following time step.

A. Distributed Coordinate Descent

Previous work has used coordinate descent for sensor control; however, that was limited to a centralized sensor network, where the central node received each sensors' pseudo-posteriors and calculated the optimal control command for each sensor. Unfortunately, there is no central node in a distributed sensor network, so that approach cannot be used. This paper proposes a new implementation of distributed coordinate descent, where the optimization is split among the sensor nodes in the network that each come to a locally optimal decision before a final decision is reached.

In further detail: After the pseudo-update step, each sensor broadcasts its set of pseudo-posteriors $\tilde{\pi}^{(i)}$ to each of its neighboring sensors $n \in \mathcal{N}^{(i)}$. Each sensor will access the pseudo-posterior densities of its neighboring sensors for each possible single-sensor control command. These pseudo-posteriors, corresponding to specific sensor control commands for each neighboring sensor, are fused to form the density that is compared to the local prediction in the objective function to calculate the reward for that set of actions.

Following the rules of coordinate descent, the variables changed are the sensor control commands for each neighboring sensor. Sequentially, one sensor's command is made variable while the rest are fixed, and the objective functions given the fused posteriors, using each possible single-sensor command

Algorithm 1 Multi-Sensor Coordinate Descent (at sensor i)

```

1: function DCD( $\{\tilde{\pi}^{(n)}\}_{n \in \mathcal{N}^{(i)}}, \mathbb{U}, m_{\max}$ )
2:   for  $m = 1 : m_{\max}$  do
3:     for  $n \in \mathcal{N}^{(i)}$  do
4:       Randomly initialize sensor control command for neighboring sensor  $n$ ,  $u_n^{(i)} \in \mathbb{U}$ .
5:     end for
6:     do
7:        $\mathbf{u}_{\text{old}}^{(i)} \leftarrow (u_1^{(i)}, \dots, u_{\mathcal{N}^{(i)}}^{(i)})$  ▷ Save the set of old control commands at start of each loop
8:       for  $n \in \mathcal{N}^{(i)}$  do
9:          $u_n^{(i)} \leftarrow \arg \max_{u \in \mathbb{U}} \nu \left( \pi_{k|k-1}^{(i)}, \text{Fusion}(\tilde{\pi}_{k,n}^{(i)}(u_n = u), \{\tilde{\pi}_{k,j}^{(i)}(u_j^{(i)})\}_{j \in \mathcal{N}^{(i)} \setminus n}) \right)$ 
          ▷ Select sensor control command that maximizes KL Divergence between  $\pi_{k|k-1}^{(i)}$  and the fused pseudo-updated densities. Updates  $u_n^{(i)}$  for next iteration to use.
10:      end for
11:      while  $\mathbf{u}_{\text{old}}^{(i)} \neq (u_1^{(i)}, \dots, u_{\mathcal{N}^{(i)}}^{(i)})$ 
12:         $\mathbf{u}_m^{(i)} \leftarrow \mathbf{u}_{\text{old}}^{(i)}$  ▷ Save set of control commands.
13:         $C_m^{(i)} \leftarrow \nu \left( \pi_{k|k-1}^{(i)}, \text{Fusion}(\{\tilde{\pi}_{k,j}^{(i)}(u_j^{(i)})\}_{j \in \mathcal{N}^{(i)}}) \right)$  ▷ Save associated reward.
14:      end for
15:      ▷ Done with  $m_{\max}$  runs, find best result.
16:       $m^{(i)*} \leftarrow \arg \min(C_m^{(i)})$ 
17:       $C^{(i)*} \leftarrow C_{m^{(i)*}}^{(i)}$ 
18:       $\mathbf{u}^{(i)*} \leftarrow \mathbf{u}_{m^{(i)*}}^{(i)}$ 
19:    return  $(\mathbf{u}^{(i)*}, C^{(i)*})$ 
20: end function

```

for the variable sensor, are calculated to find the multi-sensor control command that maximizes the reward. After each sensor's action has been optimized, the multi-sensor control command set is compared to the previously saved $\mathbf{u}_{\text{old}}$. If they are different, the cycle is rerun, as the control command set has not yet converged. If they are identical, then the coordinate descent algorithm has reached a local convergence, and the multi-sensor control command set is saved. This cycle can be stopped for computational tractability once a certain number of iterations has been reached. Finally, the coordinate descent process is run for $m_{\max}$ runs to practically ensure near-global optimization, and the set of control commands from the iteration with the highest associated reward, $\mathbf{u}^{(i)*}$, is selected. This process is shown in Algorithm 1.

After each sensor has selected its own optimal multi-sensor control command $\mathbf{u}^{(i)*}$, it broadcasts the single-sensor control command component for each individual sensor, $u_n^{(i)*}, n \in \mathcal{N}^{(i)}$, to that neighboring sensor, along with the reward $C^{(i)*}$ associated with the multi-sensor control command. Each sensor has a collection of single-sensor control commands for its action and associated rewards. The choice of how to select a final single-sensor control command from the collection at each local sensor is not trivial. Several possibilities were considered, including averaging the rewards of multiple instances of the same control command and selecting the control command with the highest average reward, summing multiple instances of the same control command before selecting the highest overall reward, and weighting each reward by the number of

neighbors the sensor that generated the command has. The method was to select the control command with the highest associated reward in the set. This is a simple way for an individual sensor's selected multi-sensor control command, if it has a high associated reward, to propagate into the network and affect the actions of neighboring sensors.

The complete proposed distributed sensor control algorithm is described in Algorithm 2, following the steps shown in Fig. 1. At each sensor locally the predicted density is calculated, pseudo-estimates are extracted from the prediction and for each possible sensor control command the PIMS and pseudo-updated density are calculated. Each set of pseudo-updates are broadcast to neighboring sensors where distributed coordinate descent described in Algorithm 1 is performed. Following coordinate descent, the pairs of selected sensor control commands and associated rewards are broadcast back to each neighboring sensor and the final decision is made locally. After the sensor control algorithm is completed the selected sensor control commands are applied, each sensor takes an action, a new set of measurements are taken, each sensor's local posterior is calculated and are fused to become the new prior densities for the following time step.

B. Objective Function

Sensor control requires either a task-dependent cost function to be minimized or an information-theoretic reward function to be maximized. Task-dependent cost functions are related to the expected error of estimation after a sensor action is

taken. Therefore the goal is to select the action that minimizes this expected error. In contrast, information-theoretic reward functions aim to quantify the expected information gain from the prior state to the posterior state after a sensor action has been taken. In this work we have chosen to use an objective function based on the Kullback-Leibler Divergence (KLD) (9), between the predicted density $\pi_{k|k-1}^{(i)}$ and the fused pseudo-updated density $\tilde{\pi}_k^{(i)}$ at sensor i as the reward function (10).

$$D_{KL}(\pi_{k|k-1}^{(i)} \parallel \tilde{\pi}_k^{(i)}) \triangleq \int \pi_{k|k-1}^{(i)}(\mathbf{X}) \log \frac{\pi_{k|k-1}^{(i)}(\mathbf{X})}{\tilde{\pi}_k^{(i)}(\mathbf{X})} \delta \mathbf{X} \quad (9)$$

KLD is a statistical measure of the difference between the distribution of interest Q and the reference distribution P . By measuring the KLD between the local predicted density and the fused pseudo-posterior of neighboring sensors, optimization of the objective function leads to a multi-sensor control command that maximizes the difference between the prediction and the fused pseudo-posterior. This exploits the structure of the probability distributions to be used for selecting the actions that minimize the future uncertainty of the target states.

$$D_{KL}(\pi_{k|k-1}^{(i)} \parallel \text{Fusion}(\tilde{\pi}_{k,n}^{(i)}(u_n = u), \{\tilde{\pi}_{k,j}^{(i)}(u_j^{(i)})\}_{j \in \mathcal{N}^{(i)} \setminus n})) \quad (10)$$

The experiments undertaken in this paper are implemented using a particle approximation of the LMB filters. This allows for computational tractability. To calculate the KLD between particle approximations of two LMB densities, the equation from Shen et al.'s work [34] is modified to suit a particle implementation:

$$D_{KL}(\pi_1 \parallel \pi_2) = \sum_{\ell \in \mathbb{L}_2} \left\{ r_1^{(\ell)} \ln \frac{r_1^{(\ell)}}{r_2^{(\ell)}} + (1 - r_1^{(\ell)}) \ln \frac{1 - r_1^{(\ell)}}{1 - r_2^{(\ell)}} + r_1^{(\ell)} D_{KL}(p_1(\cdot, \ell) \parallel p_2(\cdot, \ell)) \right\} \quad (11)$$

where

$$D_{KL}(p_1(\cdot, \ell) \parallel p_2(\cdot, \ell)) = \sum_{j=1}^{J^{(\ell)}} w_{1,j}^{(\ell)} \ln \frac{w_{1,j}^{(\ell)}}{w_{2,j}^{(\ell)}} \quad (12)$$

and $\mathbb{L}_2$ is the set of labels present in π_2 . When a target ℓ are present in $\mathbb{L}_2$ and not the set of labels present in π_1 , $\mathbb{L}_1$, then $r_1^{(\ell)} = 0$.

In sensor control applications, information on whether or not the sensors will observe the targets after taking the proposed action is significantly more important than the information about target states. Therefore, the final component of the KLD involving the spatial component of the two distributions can be ignored to significantly speed up computation time with no noticeable reduction in sensor control performance. Additionally, a penalty is added where a target is found in the predicted density and not in the pseudo-updated density, i.e. the actions result in a target no longer being observed. This

emphasizes actions that do not result in targets being dropped. This penalty takes the form

$$\sum_{\ell' \in \mathbb{L}'} \left\{ \ln \frac{1 - r_1^{(\ell')}}{1 - r_2^{(\ell')}} \right\} \quad (13)$$

where $\mathbb{L}' = \mathbb{L}_1 \setminus \mathbb{L}_2$ and $r_2^{(\ell')}$ is set to 0. The objective function used is the sum of the KLD (11), that has been modified to remove the spatial component, and the penalty (13):

$$\nu(\cdot) = \sum_{\ell \in \mathbb{L}_2} \left\{ r_1^{(\ell)} \ln \frac{r_1^{(\ell)}}{r_2^{(\ell)}} + (1 - r_1^{(\ell)}) \ln \frac{1 - r_1^{(\ell)}}{1 - r_2^{(\ell)}} \right\} + \sum_{\ell' \in \mathbb{L}'} \left\{ \ln(1 - r_1^{(\ell')}) \right\} \quad (14)$$

IV. EXPERIMENT AND RESULTS

First, the scenario that the experiments are conducted on is explained and presented. Then the different methods that are tested in this scenario are listed, followed by the results and finally, discussion and analysis of the results.

A. Scenario

The scenario consists of ten sensors tracking ten targets for 100 time-steps. The sensors are distributed uniformly in an area of 2000×2000 meters, called the sensor area and the targets can occupy any position within a 1600×1600 meter area, called the target area, with the same center as the sensor location area. Target births occur at random points along the edges of the target area, with a trajectory sampled such that the targets travel inwards from the edge of the target area. When a target reaches an edge of the target area, target death occurs. Additionally, at each time step there is a probability of survival $p_S = 0.995$ for each target, where otherwise target death occurs. The targets follow a constant turn motion model, where the turn rate θ is uniformly distributed from -0.25 to 0.25 degrees. The scenario is visualized in Fig. 2. Four of the ten targets are present initially, with the other four entering the scene over the first half of the scenario. One of the targets leaves the scene near the end of the scenario. The ground truth target cardinality is shown in Fig. 3.

Target birth is modeled with an adaptive birth model, where birth regions are created at each time step around the target observation locations from the previous time step. These birth regions have an existence probability $r = 0.04$ and standard deviations of $B = (5, 5, 5, 5, \pi/180)$.

The probability of detection for each sensor is given by the following formula:

$$p_D = p_{D_0} \frac{\tanh(\frac{d-d_2}{\lambda})}{\tanh(\frac{d_1-d_2}{\lambda})} \quad (15)$$

p_{D_0} is the probability of detection at minimum observation range (0 meters), which is set to 0.995. d_1 and d_2 are the minimum and maximum ranges where a target can be detected, respectively. These are set to 0 and 1000 meters. Additionally, each sensor's FOV is restricted to a 90 deg sector. Finally, λ is a constant that determines the shape of the curve, and is set to

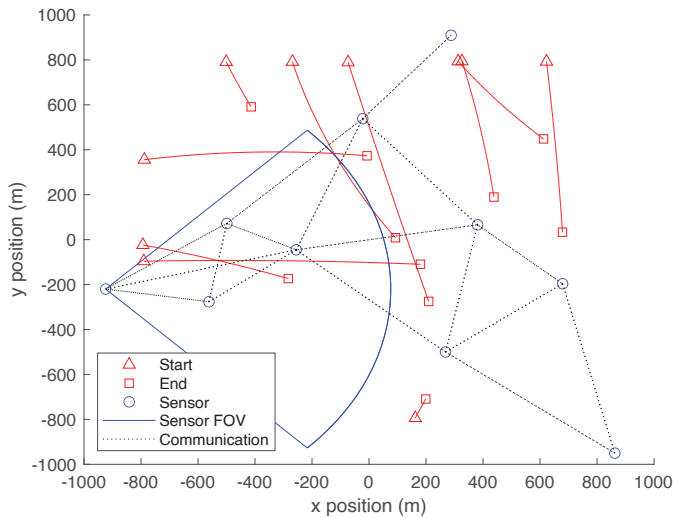


Fig. 2: Visualization of multi-target tracking scenario. The sensor locations, connections with other sensors and target paths are shown. A single sensor's FOV is depicted, with the rest omitted for clarity.

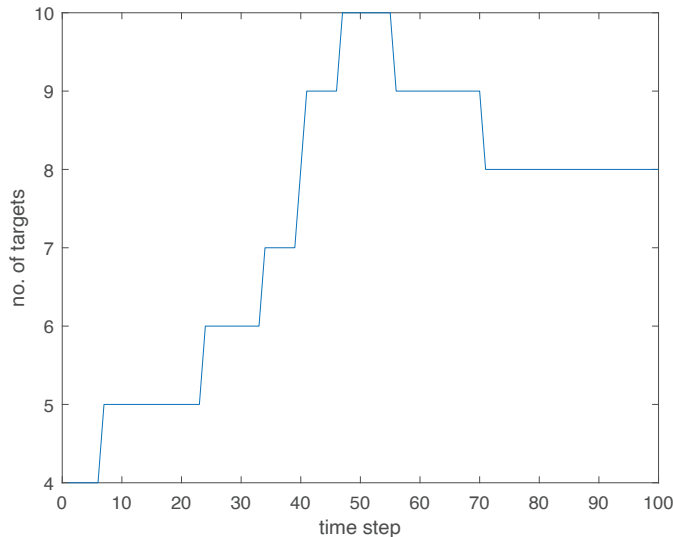


Fig. 3: The ground truth target cardinality.

65 so that $pD \approx 0.9$ when the range is 900 meters. The values of the probability of detection are shown in Fig. 4. There is distance-dependent measurement noise on each observation. Measurements are returned as a vector with two elements, the distance from the sensor to the measurement along the x-axis and y-axis, d_x and d_y respectively. The measurement noise along each axis is normally distributed, multiplied by $1 + 0.00001 * d_i^2, i \in (x, y)$.

Four methods were tested on this scenario: Fixed sensors with fusion, independent multi-sensor control with no fusion, independent multi-sensor control with fusion and multi-sensor control. For all scenarios 20 simulations were ran to perform a Monte Carlo experiment, with random initial sensor orientations. In the fixed sensor method, the sensor's

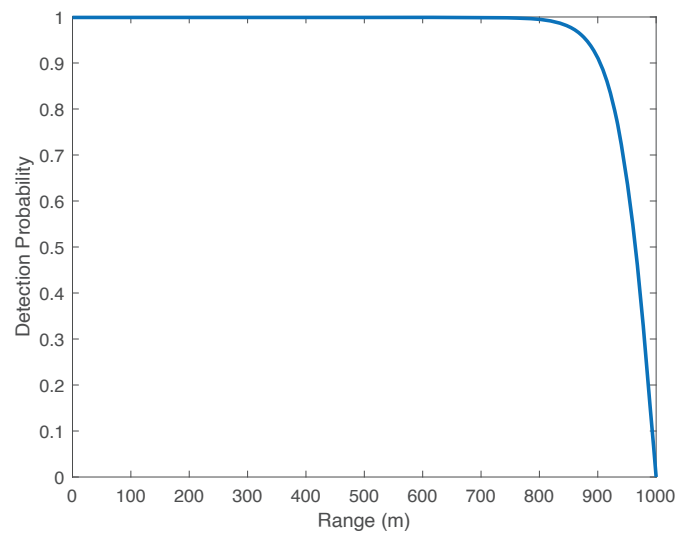


Fig. 4: The probability of detection of a target.

orientations were unchanging throughout each run. The multi-sensor control method is described in the previous section. Independent multi-sensor control refers to each sensor making control decisions independently, optimizing its own single-sensor control command, with no communication between neighboring sensors. In practice, this modifies the objective function to be (14) where π_1 remains the local predicted density $\pi_{k|k-1}^{(i)}$ and π_2 becomes the local pseudo-updated density $\tilde{\pi}_k^{(i)}$. $D_{KL}(\pi_{k|k-1}^{(i)} \| \tilde{\pi}_k^{(i)})$, at each sensor i . This independent multi-sensor control method is tested both with and without information fusion. These methods were tested to show the impact of sensor communication on performance and to determine if the proposed distributed multi-sensor control method improves upon independent multi-sensor control.

B. Results

In the experiments the OSPA distance error metric [35] is used to evaluate performance. The performance of the four methods tested via a 20 run Monte Carlo experiment on the simulated dataset is shown in Table II. The overall OSPA distance is the sum of a localization error (measured in metres) and a cardinality error (unitless) meaning that a lower value indicates more accurate target tracking; all three values are provided in Table II. The OSPA parameters used in this experiment are $c = 100$ and $p = 1$. The OSPA error of the fixed sensor method is taken as a baseline. Using the proposed independent multi-sensor control method without fusion performs poorly, failing to track the targets and underpinning the importance of information fusion in a scenario where each sensor only partially observes the scenario. Using the independent multi-sensor control method an expected improvement is observed over the baseline method. The multi-target tracking performance of the proposed distributed coordinate descent multi-sensor control method is significantly improved over all of the presented alternative methods, in both localization error and cardinality error.

	OSPA distance	OSPA localization (m)	OSPA cardinality
Fixed with fusion	27.0212	6.3738	20.6478
Independent Multi-Sensor Control without fusion	76.9432	1.3996	75.5436
Independent Multi-Sensor Control with fusion	16.2745	5.6233	10.6512
Multi-Sensor Control with fusion	11.3912	4.7822	6.6090

TABLE II: 20 run Monte Carlo results (OSPA errors averaged over all sensors and time steps) for i) fixed position sensors with fusion, ii) single-sensor control without fusion, iii) single-sensor control with fusion and iv) multi-sensor control (distributed coordinate descent) with fusion.

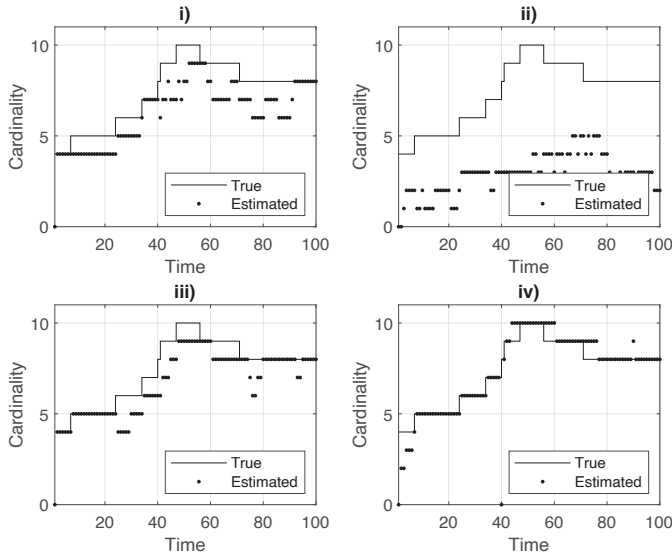


Fig. 5: Typical estimated target cardinality vs. ground truth plots. i) fixed position sensors with fusion, ii) single-sensor control without fusion, iii) single-sensor control with fusion and iv) multi-sensor control (distributed coordinate descent) with fusion.

The estimated target cardinality compared to the ground truth cardinality for a typical run from each method is shown in Fig. 5. Additionally, the OSPA distance error at each time step for a typical run from each method is shown in Fig. 6. These further show the improvement in tracking accuracy of the distributed sensor control method over the alternative methods that were tested.

V. CONCLUSION

In this work a novel algorithm for performing multi-sensor control in a distributed sensor network was proposed. This method was tested on a simulated dataset and was shown to outperform equivalent single-camera sensor control. Future work is needed to reduce the computational cost of the method to increase its practicability. Additionally, further research into alternative selection methods for determining which of the possible actions each sensor should take can potentially improve tracking performance.

ACKNOWLEDGMENT

This work was supported by the Australian Research Council through Grant DE210101181.

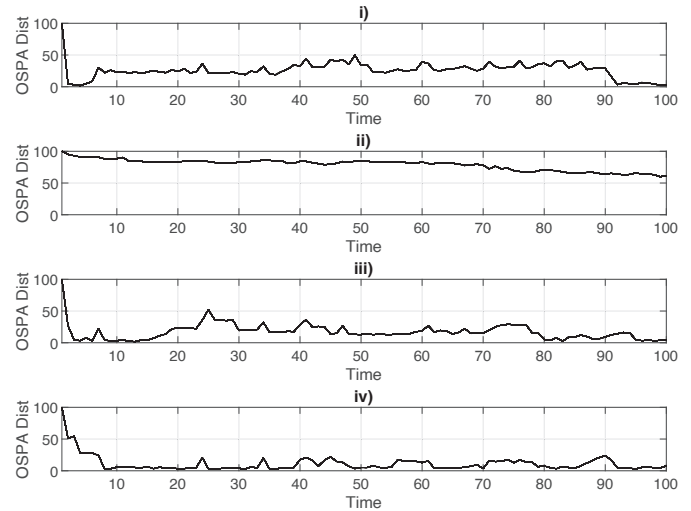


Fig. 6: Typical OSPA distance at each time step in simulation. i) fixed position sensors with fusion, ii) single-sensor control without fusion, iii) single-sensor control with fusion and iv) multi-sensor control (distributed coordinate descent) with fusion.

REFERENCES

- [1] H. H. T. Liu and B. Zhu, *Formation Control of Multiple Autonomous Vehicle Systems*. John Wiley & Sons, Ltd, 2018. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1002/9781119263081.ch4>
- [2] G. Mitchell, J. Mazurek, K. Thieriault, and P. Manghwani, *Distributed Sensor Networks*. Chapman and Hall/CRC, 2013, ch. SenSoft: Development of a Collaborative Sensor Network.
- [3] F. Lian, L. Hou, J. Liu, and C. Han, "Constrained multi-sensor control using a multi-target mse bound and a δ -glmb filter," *Sensors*, vol. 18, no. 7, 2018. [Online]. Available: <https://www.mdpi.com/1424-8220/18/7/2308>
- [4] N. Truong, U. Jayasinghe, T.-W. Um, and G. M. Lee, "A survey on trust computation in the internet of things," *THE JOURNAL OF KOREAN INSTITUTE OF COMMUNICATIONS AND INFORMATION SCIENCES (J-KICS)*, vol. 33, pp. 10–27, 01 2016.
- [5] G. Coulouris, J. Dollimore, and T. Kindberg, *Distributed Systems: Concepts and Design*, ser. International computer science series. Addison-Wesley, 2005.
- [6] C. Kreucher, A. Hero, and K. Kastella, "A comparison of task driven and information driven sensor management for target tracking," in *Proceedings of the 44th IEEE Conference on Decision and Control*, 2005, pp. 4004–4009.
- [7] A. K. Gostar, R. Hoseinnezhad, and A. Bab-Hadiashar, "Robust multi-bernoulli sensor selection for multi-target tracking in sensor networks," *IEEE Signal Processing Letters*, vol. 20, no. 12, pp. 1167–1170, 2013.
- [8] —, "Multi-bernoulli sensor control for multi-target tracking," in *2013 IEEE Eighth International Conference on Intelligent Sensors, Sensor Networks and Information Processing*, 2013, pp. 312–317.
- [9] —, "Multi-Bernoulli sensor-selection for multi-target tracking with unknown clutter and detection profiles," *Signal*

- Processing, vol. 119, pp. 28–42, 2016. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0165168415002339>
- [10] T. van Erven and P. Harremoës, “Rényi divergence and kullback-leibler divergence,” *IEEE Transactions on Information Theory*, vol. 60, pp. 3797–3820, 2014.
 - [11] R. Mahler, *Statistical Multisource-Multitarget Information Fusion*, 2007.
 - [12] B.-T. Vo, B.-N. Vo, and A. Cantoni, “The cardinality balanced multi-target multi-bernoulli filter and its implementations,” *IEEE Transactions on Signal Processing*, vol. 57, no. 2, pp. 409–423, 2008.
 - [13] B. Wei and L. Zhou, “Cell lineage tracking based on labeled random finite set filtering,” in *2018 International Conference on Control, Automation and Information Sciences (ICCAIS)*, 2018, pp. 163–168.
 - [14] N. Ishtiaq, A. K. Gostar, A. Bab-Hadiashar, and R. Hoseinnezhad, “Interaction-aware labeled multi-bernoulli filter,” 2022. [Online]. Available: <https://arxiv.org/abs/2204.08655>
 - [15] R. P. S. Mahler, “Advances in statistical multisource-multitarget information fusion,” 2014.
 - [16] L. Gao, G. Battistelli, and L. Chisci, “Fusion of labeled rfs densities with minimum information loss,” *IEEE Transactions on Signal Processing*, vol. 68, pp. 5855–5868, 2020.
 - [17] B. Ristic and B.-N. Vo, “Sensor control for multi-object state-space estimation using random finite sets,” *Automatica*, vol. 46, no. 11, pp. 1812–1818, 2010. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0005109810002955>
 - [18] A. K. Gostar, R. Hoseinnezhad, and A. Bab-Hadiashar, “Multi-bernoulli sensor control via minimization of expected estimation errors,” *IEEE Transactions on Aerospace and Electronic Systems*, vol. 51, no. 3, pp. 1762–1773, 2015.
 - [19] M. Beard, B.-T. Vo, B.-N. Vo, and S. Arulampalam, “Sensor control for multi-target tracking using cauchy-schwarz divergence,” in *2015 18th International Conference on Information Fusion (Fusion)*, 2015, pp. 937–944.
 - [20] H. G. Hoang and B. T. Vo, “Sensor management for multi-target tracking via multi-bernoulli filtering,” *Automatica*, vol. 50, no. 4, pp. 1135–1142, 2014. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0005109814000454>
 - [21] M. Jiang, W. Yi, and L. Kong, “Multi-sensor control for multi-target tracking using cauchy-schwarz divergence,” in *2016 19th International Conference on Information Fusion (FUSION)*, 2016, pp. 2059–2066.
 - [22] X. Wang, R. Hoseinnezhad, A. K. Gostar, T. Rathnayake, B. Xu, and A. Bab-Hadiashar, “Multi-sensor control for multi-object bayes filters,” *Signal Processing*, vol. 142, pp. 260–270, 2018. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0165168417302773>
 - [23] S. Panicker, A. K. Gostar, A. Bab-Hadiashar, and R. Hoseinnezhad, “Tracking of targets of interest using labeled multi-bernoulli filter with multi-sensor control,” *Signal Processing*, vol. 171, p. 107451, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S016516841930502X>
 - [24] N. Ishtiaq, S. Panicker, A. K. Gostar, A. Bab-Hadiashar, and R. Hoseinnezhad, *Selective Sensor Control via Cauchy Schwarz Divergence*. Springer, 01 2021, pp. 113–124.
 - [25] B.-N. Vo, S. Singh, A. Doucet *et al.*, “Sequential monte carlo implementation of the phd filter for multi-target tracking,” in *Proc. Int’l Conf. on Information Fusion*, 2003, pp. 792–799.
 - [26] B.-T. Vo, B.-N. Vo, and A. Cantoni, “Analytic implementations of the cardinalized probability hypothesis density filter,” *IEEE transactions on signal processing*, vol. 55, no. 7, pp. 3553–3567, 2007.
 - [27] B. Vo, B. Vo, and A. Cantoni, “On multi-bernoulli approximations to the bayes multi-target filter,” in *Proc. Int. Symp. Inf. Fusion*, 2007, pp. 71–81.
 - [28] S. Reuter, A. Scheel, and K. Dietmayer, “The multiple model labeled multi-bernoulli filter,” in *2015 18th International Conference on Information Fusion (Fusion)*. IEEE, 2015, pp. 1574–1580.
 - [29] T. Rathnayake, A. Khodadadian Gostar, R. Hoseinnezhad, R. Tennakoon, and A. Bab-Hadiashar, “On-line visual tracking with occlusion handling,” *Sensors*, vol. 20, no. 3, 2020. [Online]. Available: <https://www.mdpi.com/1424-8220/20/3/929>, ISSN = 1424-8220, DOI = 10.3390/s20030929
 - [30] S. Panicker, A. K. Gostar, A. Bab-Hadiashar, and R. Hoseinnezhad, “Sensor control for selective object tracking using labeled multi-bernoulli filter,” in *2018 21st International Conference on Information Fusion (FUSION)*, 2018, pp. 2218–2224.
 - [31] A. K. Gostar, T. Rathnayake, R. Tennakoon, A. Bab-Hadiashar, G. Battistelli, L. Chisci, and R. Hoseinnezhad, “Centralized cooperative sensor fusion for dynamic sensor network with limited field-of-view via labeled multi-bernoulli filter,” *IEEE Transactions on Signal Processing*, vol. 69, pp. 878–891, 2021.
 - [32] R. Mahler, *Advances in Statistical Multisource-Multitarget Information Fusion*, 2014.
 - [33] J. Klupacs, A. K. Gostar, T. Rathnayake, I. Gondal, A. Bab-Hadiashar, and R. Hoseinnezhad, “Multi-agent information fusion for connected driving: A review,” *IEEE Access*, pp. 1–1, 2022.
 - [34] K. Shen, C. Zhang, P. Dong, Z. Jing, and H. Leung, “Consensus-based labeled multi-Bernoulli filter with event-triggered communication,” *IEEE Transactions on Signal Processing*, vol. 70, pp. 1185–1196, 2022.
 - [35] D. Schuhmacher, B.-T. Vo, and B.-N. Vo, “A consistent metric for performance evaluation of multi-object filters,” *IEEE Transactions on Signal Processing*, vol. 56, no. 8, pp. 3447–3457, 2008.

Algorithm 2 Distributed Sensor Control

```

1: function DSC( $\mathbb{L}, \{r^{(\ell)}\}_{\ell \in \mathbb{L}}, \{\{w_j^{(\ell)}, x_j^{(\ell)}\}_{j=1}^{J^{(\ell)}}\}_{\ell \in \mathbb{L}}, B = \{r_B^{(i)}, \{w_{j,B}^{(i)}, x_{j,B}^{(i)}\}_{j=1}^{J_B^{(i)}}\}_{i \in \mathbb{B}}, p_S(x, \ell), f(x|\cdot, \ell),$ 
    $g(z|x, \ell), p_D(x, \ell), \kappa(\cdot), N_s, \{x_{s_i}\}_{i=1}^{N_s}, \mathbb{U}, m_{\max})$ 
    $\triangleright \mathbb{L}$  - Label space.
    $\triangleright \{r^{(\ell)}\}_{\ell \in \mathbb{L}}$  - current LMB existence probabilities.
    $\triangleright \{\{w_j^{(\ell)}, x_j^{(\ell)}\}_{j=1}^{J^{(\ell)}}\}_{\ell \in \mathbb{L}}$  - current LMB densities approximated by particles.
    $\triangleright B = \{r_B^{(i)}, \{w_{j,B}^{(i)}, x_{j,B}^{(i)}\}_{j=1}^{J_B^{(i)}}\}_{i \in \mathbb{B}}$  - birth model LMB parameters.
    $\triangleright p_S(x, \ell)$  - survival probability function.
    $\triangleright f(x|\cdot, \ell)$  - single-target transition density.
    $\triangleright g(z|x, \ell)$  - single-target likelihood.
    $\triangleright p_D(x, \ell)$  - detection probability.
    $\triangleright \kappa(\cdot)$  - clutter intensity.
    $\triangleright N_s$  - number of sensors.
    $\triangleright \{x_{s_i}\}_{i=1}^{N_s}$  - current sensors' states.
    $\triangleright \mathbb{U}$  - sensor control command space.
    $\triangleright m_{\max}$  - required number of consecutive local convergences for global optimization

2: for  $i = 1 : N_s$  do  $\triangleright$  Happening simultaneously in each sensor  $i$ .
3:    $\{r_+^{(\ell)}, \{w_{+j}^{(\ell)}, x_{+j}^{(\ell)}\}_{j=1}^{J_+^{(\ell)}}\}_{\ell \in \mathbb{L}_+}^{(i)} \leftarrow \text{PREDICT}(\{r^{(\ell)}, \{w_j^{(\ell)}, x_j^{(\ell)}\}_{j=1}^{J^{(\ell)}}\}_{\ell \in \mathbb{L}}, B, p_S(\cdot, \cdot), f(\cdot|\cdot, \cdot))$ 
    $\triangleright \pi_{k|k-1}^{(i)}$ . Prediction step.
4:    $\hat{X}_{pseudo}^{(i)} \leftarrow \text{ESTIMATE}(\{r_+^{(\ell)}, \{w_{+j}^{(\ell)}, x_{+j}^{(\ell)}\}_{j=1}^{J_+^{(\ell)}}\}_{\ell \in \mathbb{L}_+}^{(i)})$ 
    $\triangleright$  Extract pseudo-estimates from the prediction.
5:    $\tilde{\pi}^{(i)} \leftarrow []$ 
6:   for  $u \in \mathbb{U}$  do
7:      $Z^{(i)} \leftarrow \text{PIMS}(|\hat{X}_{pseudo}^{(i)}|, \hat{X}_{pseudo}^{(i)}, x_{s_i} + u, g(z_{s_i}|\cdot, \ell))$ 
      $\triangleright$  Generate PIMS measurements based on selected sensor control command.
8:      $\tilde{\pi}^{(i)}[u] \leftarrow \text{UPDATE}(\{r_+^{(\ell)}, \{w_{+j}^{(\ell)}, x_{+j}^{(\ell)}\}_{j=1}^{J_+^{(\ell)}}\}_{\ell \in \mathbb{L}_+}^{(i)}, Z^{(i)}, p_D(\cdot, \cdot), g(\cdot|\cdot, \cdot), \kappa(\cdot))$ 
      $\triangleright \tilde{\pi}_{k,n}^{(i)}(u)$ . Pseudo-Update the LMB parameters based on  $Z^{(i)}$ .
9:   end for
10:  end for
11:  Send sets of pseudo-updates  $\tilde{\pi}^{(i)}$  to neighboring sensors.
12:  for  $i = 1 : N_s$  do  $\triangleright$  Happening simultaneously in each sensor  $i$ .
13:     $(u^{(i)*}, C^{(i)*}) \leftarrow \text{DCD}(\{\tilde{\pi}^{(n)}\}_{n \in \mathcal{N}^{(i)}})$   $\triangleright$  Distributed Coordinate Descent.
14:  end for
15:  For each sensor  $i$ , send out  $(u_n^{(i)*}, C^{(i)*})$  to neighboring sensors  $n \in \mathcal{N}^{(i)}$ .
16:  for  $i = 1 : N_s$  do  $\triangleright$  Happening simultaneously in each sensor  $i$ .
17:     $\text{idx} \leftarrow \arg \max_{n \in \mathcal{N}^{(i)}} (C^{(n)*})$ 
18:     $u_{i,\text{final}}^{(i)} \leftarrow u_i^{(\text{idx})*}$ 
19:  end for
    $\triangleright$  Apply sensor control commands  $\{u_{i,\text{final}}^{(i)}\}_{i=1}^{N_s}$  to each sensor and acquire measurements  $\{Z^{(i)}\}_{i=1}^{N_s}$ .
20:  for  $i = 1 : N_s$  do
21:     $\{r_i^{(\ell)}, \{w_{i,j}^{(\ell)}, x_{+j}^{(\ell)}\}_{j=1}^{J_+^{(\ell)}}\}_{\ell \in \mathbb{L}_+} \leftarrow \text{UPDATE}(\{r_+^{(\ell)}, \{w_{+j}^{(\ell)}, x_{+j}^{(\ell)}\}_{j=1}^{J_+^{(\ell)}}\}_{\ell \in \mathbb{L}_+}^{(i)}, Z^{(i)}, p_D(\cdot, \cdot), g(\cdot|\cdot, \cdot), \kappa(\cdot))$ 
     $\triangleright \pi_k^{(i)}$ , Update the LMB parameters based on the measurements  $Z^{(i)}$ .
22:  end for
23:  for  $i = 1 : N_s$  do
24:     $\{\tilde{r}_i^{(\ell)}, \{\tilde{w}_{i,j}^{(\ell)}, \tilde{x}_{+j}^{(\ell)}\}_{j=1}^{J_+^{(\ell)}}\}_{\ell \in \mathbb{L}_+} \leftarrow \text{FUSION}(\{\{r_n^{(\ell)}, \{w_{n,j}^{(\ell)}, x_{+j}^{(\ell)}\}_{j=1}^{J_+^{(\ell)}}\}_{\ell \in \mathbb{L}_+}\}_{n \in \mathcal{N}^{(i)}})$ 
     $\triangleright \tilde{\pi}_k^{(i)}$ , Distributed Fusion.
25:  end for
26:  return  $\{\tilde{r}_i^{(\ell)}, \{\tilde{w}_{i,j}^{(\ell)}, \tilde{x}_{+j}^{(\ell)}\}_{j=1}^{J_+^{(\ell)}}\}_{\ell \in \mathbb{L}_+}$ 
    $\triangleright$  Return fused and updated density ready for the next time step.
27: end function

```
